ChatGPT

zWHy2020/Multimodal_Compress_- 代码库深研与 ICCV2025 InstanceDepth 论文对齐分析报告

执行摘要

该 GitHub 项目当前的核心目标是：在**无线信道噪声（AWGN/衰落）**等可微仿真信道中，实现**深度图（Depth）+ 视频（Video）**的端到端联合传输/压缩（Deep JSCC 风格），并通过“共享潜变量 + 私有残差”的跨模态分解、熵模型码率估计、互信息（MI）校正与 OMIB-like KL 正则，形成一个可训练的多模态系统。系统的统一输出接口通过 `TypedDict` 固化为 `ModelForwardOutput`，便于训练脚本与损失函数解耦。 [1]

从工程结构看，仓库可分为两层：
- **核心子包 `modules/`（11 个 Python 模块）**负责可插拔的系统编排、信道、深度/视频编解码、融合与熵模型、带宽门控与 VQ 量化等。
- **仓库根目录（15 个 Python 模块）**提供训练入口、配置、数据加载、损失/指标、视频细节实现、工具与判别器等。

与用户提供的 ICCV2025 论文（`Instance-Level Video Depth in Groups Beyond Occlusions`）相比：论文侧提出 **InstanceDepth 两阶段方法**（Holistic Depth Initialization → Instance-Aware Depth Rectification）与 **GID 数据集**（101,500 帧、实例级深度标注），用于“群体/遮挡场景下实例级视频深度”估计。 [2]
本仓库并不实现该论文的深度估计网络，而更偏向**把“深度图 + 视频”作为待传输源**来做端到端 JSCC/语义通信式压缩；二者的结合点更自然地落在数据与评测：用 GID 的真实遮挡/群体视频与实例深度标注，构造本仓库的 depth/video 输入来测试在复杂遮挡场景下的鲁棒传输质量（而非复现论文网络）。 [3]

> 说明：该环境可访问 CVF Open Access 的论文摘要页，但对 PDF 正文抓取/渲染多次失败（因此无法逐式引用论文中公式与章节细节）。本报告关于 ICCV2025 的技术细节仅基于其 CVF 摘要页可见信息与其公开指向的资源，不对 PDF 正文内容作未经核验的推断。 [4]

仓库模块数量与模块对比表

模块数量结论

以 **main 分支可读取的 Python 源文件**为准，本项目目前包含：

- **核心子包 `modules/`：11 个模块**
  `api.py, system.py, channel_models.py, depth_codec.py, depth_models.py, fusion.py, fusion_models.py, video_codec.py, gating.py, quantization.py, __init__.py`。
- **根目录：15 个模块**
  `train.py, config.py, data_loader.py, losses.py, metrics.py, channel.py, video_encoder.py, video_unet.py, multimodal_jscc.py, discriminator.py, utils.py, mine_utils.py, cross_attention.py, utils_check.py, __init__.py`（其中 `utils_check.py` 更偏诊断/兼容性脚本）。

合计 **26 个 Python 模块**（不含可能存在的非 Python 资源文件；本报告不对未读取到的文件做臆测）。

模块对比表

下表采用“功能模块（非逐文件）”的粒度，以便直接对应任务中的“purpose / inputs / outputs / run”：

模块名	路径（主入口）	目的	主要输入	主要输出
数据加载与批处理	`data_loader.py`	读取 Manifest v1/v2，按策略抽帧并组 batch；支持严格模式丢弃缺失样本	`manifest.json`、视频文件、深度图文件	`batch={"inputs","targ` 含 `video_frame_mask`
核心系统编排（Depth+Video JSCC）	`modules/system.py`	组合深度/视频编码、跨模态分解、信道仿真、解码，并产出码率与正则统计	`depth_input [B,1,H,W]`，`video_input [B,T,3,H,W]`，`snr_db`	`ModelForwardOutput`：rate_stats、omib_stats
跨模态融合与共享/私有分解	`modules/fusion.py`	把 depth/video 特征压缩为共享向量，并回投影得到私有残差	depth_feat/video_feat	`FusionOutput`（`shared`
熵模型与码率估计	`modules/fusion.py`（`JointEntropyModel`）	用高斯先验 NLL 估计 bits-per-element（BPE）并汇总 joint/shared/private	shared/private latents	`entropy_stats` + `rate`
信道建模与噪声注入	`channel.py`	AWGN / Rayleigh / Rician 信道仿真；可更新 SNR；含功率归一化	任意张量特征	加噪后特征
视频编码器（MDVSC 风格）	`video_encoder.py`	GOP 潜空间变换→JSCC 编码→共性/个性特征→熵分数掩码（target CBR）	`video_frames [B,T,C,H,W]`，`snr_db`，`gop_size` 等	`encoded_features [B,T(+1),C,H’,W’]`，`guide_vectors [B,T,d]`

模块名	路径（主入口）	目的	主要输入	主要输出
视频解码器（U-Net）	`video_unet.py`	将带噪潜变量恢复为视频帧；支持分块解码与输出裁剪	`noisy_features [B,T,C,H,W]`	`video_decoded [B,T,3,`
深度编解码（轻量 CNN）	`modules/depth_codec.py`	深度图→潜变量→深度图重建；输出 guide 向量	depth map tensor	depth latent + depth de
带宽门控与量化（可选能力）	`modules/gating.py` / `modules/quantization.py`	通道维裁剪/FiLM gate；VQ 离散码本与经验熵	features + (snr, ratio)	gated features；VQ ind
训练、优化与评估指标	`train.py` / `losses.py` / `metrics.py`	训练循环、DDP/AMP、SNR curriculum；RD+感知+几何损失；PSNR/SSIM/深度指标	`ModelForwardOutput` + targets	loss dict、metrics dict、

表中每一行的“目的/输入/输出”都可在对应文件的注释、函数签名与数据结构中直接验证（见下一节的关键代码片段与路径引用）。

模块级分析与可运行性

统一数据接口与系统输出约定

项目用 `modules/api.py` 约定模块间数据结构，其中 `ModelForwardOutput` 规定了前向输出可包含 `depth_decoded/video_decoded`、`rate_stats` 与 `omib_stats` 等字段，训练脚本与损失函数依赖这些键做计算。[1]

关键片段（节选）：

```
# modules/api.py
class RateStats(TypedDict, total=False):
    joint_bpe: torch.Tensor
    shared_bpe: torch.Tensor
    depth_private_bpe: torch.Tensor
    video_private_bpe: torch.Tensor
    cross_modal_mi_bits: torch.Tensor

class ModelForwardOutput(TypedDict, total=False):
    depth_decoded: torch.Tensor
```

```
    video_decoded: torch.Tensor
    entropy_stats: Dict[str, torch.Tensor]
    rate_stats: RateStats
    omib_stats: Dict[str, torch.Tensor]
```

[1]

这使得系统的“模块化边界”非常清晰：**模型前向输出 = 可视重建 + 可微码率估计 + 正则统计**，损失侧只需要读字典。

核心系统 `DepthVideoJSCC` 的输入/输出与依赖

`modules/system.py` 的 `DepthVideoJSCC` 明确要求同时输入 depth 与 video（否则抛错），并输出包含编码、中间潜变量、传输后潜变量、重建结果、码率与 OMIB/MI 统计的字典。[5]

关键片段（节选）：

```
# modules/system.py
if depth_input is None or video_input is None:
    raise RuntimeError('DepthVideoJSCC 联合压缩需要同时输入 depth_input 和 video_input。')

depth_encoded, depth_guide = self.depth_encoder(depth_input)
video_encoded, video_guide = self.video_encoder(video_input)

fused = self.joint_fusion(depth_encoded, video_encoded)
entropy_stats = self.entropy_model(fused['shared_latent'], fused['depth_private'],
fused['video_private'])

shared_tx = self.channel(self._norm_feature(fused['shared_latent'], 'shared'))
depth_private_tx = self.channel(self._norm_feature(fused['depth_private'], 'depth'))
video_private_tx = self.channel(self._norm_feature(fused['video_private'], 'video'))

out['depth_decoded'] = self.depth_decoder(depth_latent_rx, depth_guide)
out['video_decoded'] = self.video_decoder(video_latent_rx, video_guide, output_size=...)
```

[5]

依赖层面（通过 import 可见）：
- 深度编解码：`modules/depth_models.py`，`modules/depth_codec.py`。
- 视频编解码：`modules/video_codec.py` 依赖根目录 `video_encoder.py` / `video_unet.py`。
- 融合/熵模型/MI：`modules/fusion_models.py` 与 `modules/fusion.py`。
- 信道：`modules/channel_models.py` →根目录 `channel.py`。

视频子系统：MDVSC 风格的视频语义通信编码器

`video_encoder.py` 的文件头部明确宣称其“基于 MDVSC 的视频语义通信架构”，并实现了 GOP 级潜空间变换、共性/个性特征提取与熵模型掩码。[6]
其关键约束之一是：输入帧数 `T` 必须能被 `gop_size` 整除，否则直接报错；训练脚本中也专门加入了 GOP 对齐逻辑。

节选：

```
# video_encoder.py
if T % self.gop_size != 0:
    raise ValueError(f"video_frames长度T={T}不能被gop_size={self.gop_size}整除。")

entropy_scores = self.entropy_model(common_feature, individual_features)
entropy_mask = self.entropy_model.build_mask(entropy_scores, target_cbr=self.target_cbr,
training=self.training)
```

[6]

研究对应关系：该类结构与“视频语义通信/可控码率”的近期工作脉络一致（例如 MDVSC 以“共性/个性语义特征 + 熵模型可控”作为核心思想）。[7]

数据与运行方式

数据格式（Manifest v2）与严格模式

`data_loader.py` 注释给出 Manifest v2 的组织方式：**“每视频一条记录，包含 depth/video 资源”**，并在 `__getitem__` 中要求 `item["depth"]["file"]` 与 `item["video"]["file"]` 存在；严格模式下关键模态缺失将丢弃样本并返回 `_dropped` 原因。[8]

批处理输出被整理为：

- `inputs["depth_input"] = depths`、`targets["depth"] = depths`
- `inputs["video_input"] = videos`、`targets["video"] = videos`
- `inputs["video_frame_mask"]` 为对短视频 repeat-last padding 的有效帧掩码。[8]

训练入口与命令行参数

训练脚本入口为 `train.py`，必须提供 `--data-dir`；若未显式指定 manifest，会优先寻找 `train_manifest_v2.json/val_manifest_v2.json`，否则回退到 v1 名称。[9]

节选：

```
# train.py
parser.add_argument('--data-dir', type=str, required=True, help='数据目录')
parser.add_argument('--train-manifest', type=str, default=None, help='训练数据清单路径（相对于
data-dir）')
...
if args.train_manifest:
    config.train_manifest = os.path.join(config.data_dir, args.train_manifest)
else:
    default_train_v2 = os.path.join(config.data_dir, "train_manifest_v2.json")
    config.train_manifest = default_train_v2 if os.path.exists(default_train_v2) else ...
```

[9]

可运行示例（以仓库现有 CLI 为准）：

```
python train.py --data-dir /path/to/data
# 或显式指定
python train.py --data-dir /path/to/data --train-manifest train_manifest_v2.json --val-manifest val_manifest_v2.json
```

[9]

自测与测试覆盖情况

项目未发现标准化单元测试框架（如 `pytest`）入口；但 `data_loader.py` 提供了一个最小自测 main：随机取两条样本并打印形状。 [8]

```
python data_loader.py --manifest /path/to/manifest.json --data_dir /path/to/data --video_sampling_strategy uniform
```

[8]

整体架构、数据流与数学映射

端到端数据流与组件分工

从 `DepthVideoJSCC.forward()` 可抽象出如下端到端链路：

- **源（Source）**：`depth_input`（单通道深度图）与 `video_input`（视频帧序列）。
- **编码（Encoder）**：
- Depth：轻量 CNN 降采样编码 + global guide。 [10]
- Video：MDVSC 风格 GOP 潜空间变换 + JSCC encoder + 共性/个性特征 + 熵掩码（target CBR）。 [6]
- **跨模态联合（Fusion）**：全局池化后拼接→共享向量 `shared_latent`，再线性投影回各模态形成“共享部分”，剩余为“私有残差”。
- **码率估计（Entropy Model）**：对共享/私有潜变量用高斯 NLL（nat→bits）估计 BPE，并聚合为 `joint_bpe`。
- **信道（Channel）**：对 `shared_latent / depth_private / video_private` 分别进行功率归一化与 AWGN/衰落模拟。
- **解码（Decoder）**：
- Depth：转置卷积恢复到深度图（Sigmoid 输出）。 [10]
- Video：U-Net 在潜空间恢复帧序列（再映射到 [0,1]，可选归一化输出）。 [11]
- **统计与正则**：
- MI 校正（MINE DV 下界）用于 `rate_stats["cross_modal_mi_bits"]`。
- OMIB-like：用私有潜变量的经验均值/方差近似高斯，计算 `KL(q||N(0,I))`。 [5]

架构图（Mermaid）

```
flowchart TD
  A[Manifest v2 JSON] --> B[data_loader.py\n抽帧+严格模式+batch]
  B --> C1[depth_input\nB×1×H×W]
  B --> C2[video_input\nB×T×3×H×W]
```

```
C1 --> D1[DepthJSCCEncoder\nmodules/depth_codec.py]
C2 --> D2[VideoJSCCEncoder (MDVSC)\nvideo_encoder.py]

D1 --> E1[depth_feat + depth_guide]
D2 --> E2[video_feat + video_guide]

E1 --> F[JointLatentFusion\nshared+private\nmodules/fusion.py]
E2 --> F

F --> G[JointEntropyModel\nBPE估计\nmodules/fusion.py]
F --> H[Channel模拟\nAWGN/Rayleigh/Rician\nchannel.py]

H --> I[shared_tx/depth_tx/video_tx]
I --> J[重组潜变量\nshared->modal + private]
J --> K1[DepthJSCCDecoder\nmodules/depth_codec.py]
J --> K2[VideoUNetDecoder\nvideo_unet.py]

K1 --> L1[depth_decoded]
K2 --> L2[video_decoded]

G --> M[rate_stats]
F --> N[MINE/OMIB统计\nmodules/fusion.py\nmodules/system.py]

L1 --> O[DepthVideoLoss\nlosses.py]
L2 --> O
M --> O
N --> O

O --> P[Adam + LR Scheduler\ntrain.py]
```

与经典 JSCC / RD 公式的对应

Deep JSCC 的端到端映射

深度 JSCC 的基本形式可以概括为：
编码器把源信号映射到信道输入 `w = E_θ(x)`，信道 `y = η(w) + n`，解码器恢复 `x' = D_ϕ(y)`，整体为 `x' = D_ϕ(η(E_θ(x)))`。在一篇可公开获取的综述性工程论文（MDPI Electronics 2023）中，这一链路被显式写为 JSCE/JSCD 的组合形式，并给出 AWGN 等模型的可微建模。[12]

本仓库的对应关系（严格以代码为准）为：
- `E_θ`：`DepthJSCCEncoder + VideoJSCCEncoder + JointLatentFusion`（其中视频编码器内部还包含 GOP latent transform 与 entropy mask）。
- `η + n`：`Channel/AWGNChannel`（或 Rayleigh/Rician）对潜变量加噪。
- `D_ϕ`：`DepthJSCCDecoder + VideoUNetDecoder`。

RD（Rate–Distortion）目标与码率代理

神经压缩/端到端压缩通常以 **率失真权衡**为核心：最小化 `D + λR`，其中 `R` 常由熵模型（对潜变量的对数似然）近似。Ballé 等人在神经图像压缩（scale hyperprior）中系统化了这种“可训练熵模型 + RD 优化”的范式。[13]

在本仓库中：
- `D` 由 `DepthLoss`（SILog + edge + normal）与 `VideoLoss`（Charbonnier + temporal + 可选 perceptual）组成。
- `R` 由 `JointEntropyModel._nll_bits` 对共享/私有潜变量的高斯 NLL（bits）计算得到，并以均值形式输出为 BPE。

其高斯 NLL→bits 的实现非常直接：

```
# modules/fusion.py
nll_nat = 0.5 * ((x / scale) ** 2) + torch.log(scale) + 0.5 * math.log(2 * math.pi)
return nll_nat / math.log(2.0)  # nat -> bits
```

[14]

互信息（MI）校正与 OMIB-like 正则的数学对应

MINE（Donsker–Varadhan）下界

MINE 的核心是用 Donsker–Varadhan 表示来给出 KL/MI 的变分下界：
`D_KL(P||Q) = sup_T E_P[T] - log E_Q[e^T]`，从而构造 `I(X;Y)` 的可训练下界估计器。[15]

本仓库在 `modules/fusion.py` 中实现 DV 形式：

```
# modules/fusion.py
t_joint = self.net(joint)
t_marginal = self.net(marginal)
mi_nat = t_joint.mean() - torch.log(torch.exp(t_marginal).mean().clamp_min(1e-8))
```

[14]

并在 `modules/system.py` 中把 MI（nat→bits）加入 `rate_stats["cross_modal_mi_bits"]`，随后在损失里用 `rate_penalty - mi_correction` 将其作为“修正项”进入 RD 部分。

OMIB-like：KL(q||N(0,I)) 的 VAE 风格近似

`modules/system.py` 以私有潜变量的经验均值/方差近似高斯后验 `q(z)`，并计算 `KL(q(z)||N(0,I))` 的逐元素形式（典型 VAE 推导）。这与 VAE 文献中通过 KL 项正则潜变量的做法一致（Auto-Encoding Variational Bayes）。[16]

代码片段（节选）：

```
# modules/system.py
depth_mu = fused['depth_private'].mean(dim=(2, 3))
depth_var = fused['depth_private'].var(dim=(2, 3), unbiased=False).clamp_min(self.omib_eps)
depth_logvar = torch.log(depth_var)
depth_kl = 0.5 * (depth_mu.pow(2) + depth_var - 1.0 - depth_logvar)
```

[5]

损失侧 `DepthVideoLoss` 把 `depth_kl + r * video_kl` 乘以 `beta_eff` 加入总损失；`beta_eff` 可由 MINE 估计到上界（`train.py` 的 `--use-mine-beta-bound` 路径），并写入配置的 `ib_beta_max`。

关键未明确之处（按“不可臆测”原则标注）

- **数据来源/数据集名称与预处理生成深度图的方式**：仓库给出 Manifest 结构、加载与训练流程，但未提供特定数据集下载与“深度图如何生成/标注”的说明，因此数据集制作步骤属未指定。
- **量化与带宽门控是否参与默认训练**：仓库提供 `BandwidthMask/ConditionalBandwidthGate/VectorQuantizer`，但 `DepthVideoJSCC`（modules/system.py）当前 forward 并未调用这些组件；训练脚本只在模型存在 `set_bandwidth_ratio` 时尝试设置 ratio，而当前 `DepthVideoJSCC` 未实现该方法，因此“带宽退火”对核心系统是否生效需额外工程接入才能确定。

训练与推理管线、优化细节与部署考量

训练管线（train.py）

训练脚本实现了完整 train/val 循环，并支持：
- **SNR 策略**：`random / curriculum / fixed`，默认在 `TrainingConfig` 中设为 `curriculum`，并在每个 batch 取当前 snr_db。
- **AMP 混合精度 + GradScaler**（CUDA 条件）。
- **梯度累积与裁剪**（`gradient_accumulation_steps`，`grad_clip_norm`）。
- **分布式 DDP**（`--distributed` 或环境变量 `WORLD_SIZE/LOCAL_RANK`）。[9]
- **学习率调度器**：cosine/step/plateau（默认 cosine）。
- **可选对抗训练**：启用判别器 PatchGAN、hinge loss、可选 R1 正则。

对抗训练侧的“R1 梯度惩罚”与“hinge 对抗”在 GAN 稳定训练中常见，Mescheder 等对 GAN 收敛与正则项做过系统分析（R1 在相关工作中广泛使用）。[17]

损失函数栈（losses.py）

当前仓库的设计明显偏向“RD + 感知 + 几何”三类项的可组合：
- 视频：Charbonnier 重建 + temporal（帧差一致）+（可选）LPIPS/VGG 感知、3D 时空特征损失。[18]
LPIPS 的理论背景对应 Zhang 等提出的“深特征感知度量”工作。[19]
- 深度：SILog + 梯度一致 + 法向一致。[18]
SILog 与“尺度不变深度误差”的经典做法可追溯到 Eigen 等的深度预测论文。[20]
- 码率：来自熵模型 bpe；并引入动态权重 `r` 平衡 depth/video 的 rate penalty。
- MI 校正：以 bits 形式从 rate penalty 中扣除（注意：该项在信息论意义上属于“训练启发式正则/修正”，并非标准 RD 推导）。
- OMIB-like：`beta_eff * (KL_depth + r * KL_video)`。

推理与部署考量（从代码可直接推出的约束）

- **输入必须同时包含深度与视频**：当前 `DepthVideoJSCC` 直接拒绝缺失任一模态。若希望部署时可单模态工作，需要修改该逻辑或提供缺失模态填充路径。
- **视频帧数/GOP 对齐**：`VideoJSCCEncoder` 要求 `T % gop_size == 0`，训练脚本会自动把 `video_gop_size` 对齐到 `video_clip_len` 的约数；部署时也应保证同样约束，否则推理报错。
- **信道建模是“张量级可微噪声层”**：`AWGNChannel` 的噪声功率直接由 `noise_power = 1/(10^(snr_db/10))` 设置，并在 forward 中对输入张量加噪；部署到真实无线链路时，还需定义“如何把这些潜变量映射到真实调制符号/带宽资源”的工程接口，本仓库当前未提供物理层 I/Q 映射与协议栈对接。
- **带宽控制（CBR/ratio）存在两条路径但未完全贯通**：
- 视频编码器内部有 `target_cbr` 并用熵分数阈值构造掩码（硬/软直通估计）。 [6]
- 通用门控模块 `ConditionalBandwidthGate/BandwidthMask` 也可做通道维裁剪与 FiLM 缩放。 [21] 但核心 `DepthVideoJSCC` forward 并未调用上述门控/量化模块，且训练脚本仅在模型实现 `set_bandwidth_ratio` 时注入 ratio，这意味着“可控码率/带宽退火”要在系统编排层显式接入才会真正生效。

ICCV2025 InstanceDepth 论文与本仓库的对齐空间

论文可核验贡献点（基于 CVF 摘要页）

ICCV2025 论文在 CVF 开源页给出的可核验信息包括：

- 提出 **GID**：一个面向真实活动场景（real-world activity scenes）的**大规模视频深度数据集**，包含 **101,500 帧**，并提供**实例级深度标注**，用于“视频中群体实例深度估计”。 [2]
- 提出 **InstanceDepth**：两阶段方法
- **Holistic Depth Initialization**：先估计整体深度
- **Instance-Aware Depth Rectification**：再结合实例掩码/先验进行修正，以提升遮挡下实例深度质量，并强调可从遮挡观测中推理被遮挡区域。 [2]
- 提供项目主页/代码与数据集地址（CVF 页直接给出 GitHub 链接指向 `ViktorLiang/GID`）。 [2]

由于 PDF 正文未能在本环境成功抓取，论文中的网络结构细节、损失项公式、训练超参与 ablation 表格无法逐项核验，因此本报告不对其做未经证实的复述。 [22]

与本仓库的工程结合点（明确标注为“推断/建议”，非论文或代码事实）

在不臆测论文正文的前提下，二者的最直接结合点是“数据与评测”而非“模型复现”：

- 本仓库的数据加载器要求同时提供 `video.file` 与 `depth.file`，而 GID 明确提供视频帧与实例级深度标注（可以导出/渲染为单通道深度图）。因此，**可用 GID 构造本仓库所需的 depth+video 配对样本**，以研究“在强遮挡/群体场景下的鲁棒传输与重建质量退化曲线”。（推断依据：GID 提供实例级深度标注与视频帧；本仓库以深度图+视频为输入训练/重建。） [3]
- 论文强调遮挡与实例形状先验；本仓库当前并未传输实例 mask/shape prior。但从通信角度，可以把“实例掩码/语义先验”作为额外模态或 side information（类似现代编解码器的 side info/超先验思想），从而潜在提高遮挡场景下的重建质量。这属于研究方向建议，并不由当前代码直接支持。 [23]

参考与链接索引

GitHub 仓库与关键文件链接

```
Repo:
https://github.com/zWHy2020/Multimodal_Compress_-

核心接口:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/api.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/system.py

跨模态融合/熵模型/MI:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/fusion.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/fusion_models.py

信道:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/channel.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/channel_models.py

深度编解码:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/depth_codec.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/depth_models.py

视频编解码:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/video_encoder.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/video_unet.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/video_codec.py

带宽门控与量化:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/gating.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/quantization.py

训练/配置/数据/损失/指标:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/train.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/config.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/data_loader.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/losses.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/metrics.py

判别器与工具:
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/discriminator.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/utils.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/mine_utils.py
```

ICCV2025 论文与公开资源链接

```
CVF Open Access 论文页（含摘要、PDF入口、代码/数据指向）:
https://openaccess.thecvf.com/content/ICCV2025/html/Liang_Instance-
Level_Video_Depth_in_Groups_Beyond_Occlusions_ICCV_2025_paper.html
```

```
用户提供的 PDF 链接（本环境抓取失败，但仍给出原始链接）：
https://openaccess.thecvf.com/content/ICCV2025/papers/Liang_Instance-
Level_Video_Depth_in_Groups_Beyond_Occlusions_ICCV_2025_paper.pdf

CVF 页面指向的代码/数据集仓库（GID）：
https://github.com/ViktorLiang/GID
```

2

本报告引用的关键理论/论文链接

```
Deep JSCC（无线图像端到端 JSCC）：
https://discovery.ucl.ac.uk/id/eprint/10089480/   (UCL Discovery 条目，可下载 PDF)

JSCC 链路公式与可微信道建模（含 JSCE/JSCD 表达式）：
https://www.mdpi.com/2079-9292/12/22/4637  (MDPI Electronics 2023)

神经压缩 RD 范式（Scale Hyperprior）：
https://research.google/pubs/variational-image-compression-with-a-scale-hyperprior/
https://arxiv.org/abs/1802.01436

MINE（Donsker–Varadhan 形式的 MI 下界）：
https://arxiv.org/abs/1801.04062

VAE 与 KL 正则（Auto-Encoding Variational Bayes）：
https://arxiv.org/abs/1312.6114

SILog/尺度不变深度误差的经典来源（Eigen et al.）：
https://papers.nips.cc/paper/2014/file/7bccfde7714a1ebadf06c5f4cea752c1-Paper.pdf

LPIPS 感知损失的来源:
https://openaccess.thecvf.com/content_cvpr_2018/html/
Zhang_The_Unreasonable_Effectiveness_CVPR_2018_paper.html
https://arxiv.org/abs/1801.03924

MDVSC（视频语义通信，代码中点名的体系来源之一）：
https://arxiv.org/abs/2305.15799
https://arxiv.org/abs/2308.05338

DVST / Wireless Deep Video Semantic Transmission（相关视频语义通信脉络）：
https://arxiv.org/abs/2205.13129
```

24

---

1 api.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/api.py

2 3 4 https://openaccess.thecvf.com/content/ICCV2025/html/Liang_Instance-Level_Video_Depth_in_Groups_Beyond_Occlusions_ICCV_2025_paper.html
https://openaccess.thecvf.com/content/ICCV2025/html/Liang_Instance-Level_Video_Depth_in_Groups_Beyond_Occlusions_ICCV_2025_paper.html

5 system.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/system.py

6 video_encoder.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/video_encoder.py

7 https://dblp.org/rec/journals/corr/abs-2305-15799
https://dblp.org/rec/journals/corr/abs-2305-15799

8 data_loader.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/data_loader.py

9 train.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/train.py

10 depth_codec.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/depth_codec.py

11 video_unet.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/video_unet.py

12 https://www.mdpi.com/2079-9292/12/22/4637
https://www.mdpi.com/2079-9292/12/22/4637

13 23 https://research.google/pubs/variational-image-compression-with-a-scale-hyperprior/
https://research.google/pubs/variational-image-compression-with-a-scale-hyperprior/

14 fusion.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/fusion.py

15 https://www.researchgate.net/publication/322498528_MINE_Mutual_Information_Neural_Estimation
https://www.researchgate.net/publication/322498528_MINE_Mutual_Information_Neural_Estimation

16 https://huggingface.co/papers/1312.6114
https://huggingface.co/papers/1312.6114

17 https://is.mpg.de/avg/publications/meschedericml2018
https://is.mpg.de/avg/publications/meschedericml2018

18 losses.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/losses.py

19 https://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_The_Unreasonable_Effectiveness_CVPR_2018_paper.html
https://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_The_Unreasonable_Effectiveness_CVPR_2018_paper.html

20 https://papers.nips.cc/paper/5539-depth-map-prediction-from-a-single-image-using-a-multi-scale-deep-network.pdf
https://papers.nips.cc/paper/5539-depth-map-prediction-from-a-single-image-using-a-multi-scale-deep-network.pdf

21 gating.py
https://github.com/zWHy2020/Multimodal_Compress_-/blob/main/modules/gating.py

22 (no title)

24 https://discovery.ucl.ac.uk/id/eprint/10089480/
https://discovery.ucl.ac.uk/id/eprint/10089480/